

O **Path** do **FastAPI** pode atuar de várias formas nas requisições do usuário, especialmente na validação de parâmetros de rota. Aqui estão algumas atribuições onde ele pode ser útil:

◆ 1. Validação de Intervalos Numéricos

Permite restringir os valores aceitáveis para um parâmetro de rota.

PYTHON

```
from fastapi import FastAPI, Path
```

```
app = FastAPI()
```

```
@app.get("/itens/{item_id}")
async def get_item(item_id: int = Path(ge=1, le=100, title="ID do Item", description="Deve estar entre 1 e 100")):
    return {"item_id": item_id}
```

❑ **Aceita:** /itens/10

❑ **Rejeita:** /itens/0 ou /itens/101 (fora do intervalo)

◆ 2. Definição de Um Valor Obrigatório

O **Path** pode ser usado para garantir que um valor específico seja informado.

PYTHON

```
@app.get("/usuarios/{user_id}")
async def get_user(user_id: int = Path(..., title="ID do Usuário")):
    return {"user_id": user_id}
```

Aqui, `...` significa que o parâmetro é **obrigatório**.

◆ 3. Adição de Metadados para a Documentação

Melhora a descrição dos parâmetros na **Swagger UI** e no **ReDoc**.

PYTHON

```
@app.get("/produtos/{produto_id}")
async def get_produto(produto_id: int = Path(title="ID do Produto", description="Código identificador do produto")):
    return {"produto_id": produto_id}
```

Isso aparece automaticamente na **documentação interativa** gerada pelo FastAPI.

◆ 4. Definição de Valores Padrão (Mas Obrigatórios)

Embora o `Path` sempre exija um valor, ele pode ter um valor padrão que será validado.

PYTHON

```
@app.get("/categorias/{categoria_id}")
async def get_categoria(categoria_id: int = Path(default=1, ge=1,
le=5, title="ID da Categoria")):
    return {"categoria_id": categoria_id}
```

Se o usuário não passar um `categoria_id`, o padrão será 1, mas ele ainda precisa estar entre 1 e 5.

◆ 5. Uso de Expressões Regulares para Validação

Podemos restringir o formato dos valores passados na rota.

PYTHON

```
from fastapi import Path

@app.get("/pedidos/{codigo_pedido}")
async def get_pedido(codigo_pedido: str = Path(regex="^PED[0-9]{4}$",
title="Código do Pedido")):
    return {"codigo_pedido": codigo_pedido}
```

☐ **Aceita:** /pedidos/PED1234

☐ **Rejeita:** /pedidos/1234 ou /pedidos/ABC123 (não segue o padrão PED0000)

◆ 6. Aceitar Somente Certos Valores Específicos

Podemos limitar um parâmetro de rota para aceitar apenas alguns valores.

PYTHON

```
@app.get("/status/{status_id}")
async def get_status(status_id: int = Path(title="Status",
description="1=Ativo, 2=Inativo, 3=Em análise", le=3, ge=1)):
    return {"status_id": status_id}
```

☐ **Aceita:** /status/1, /status/2, /status/3

☐ **Rejeita:** /status/0, /status/4 (fora da faixa)

◆ 7. Usar Path em Conjunto com Query

Podemos misturar Path (parâmetros na URL) com Query (parâmetros opcionais).

PYTHON

```
from fastapi import Query

@app.get("/clientes/{cliente_id}")
async def get_cliente(cliente_id: int = Path(ge=1), ativo: bool =
Query(default=True)):
    return {"cliente_id": cliente_id, "ativo": ativo}
```

❑ **Acesso válido:** /clientes/2?ativo=false

◆ 8. Diferenciar Parâmetros de Path e Query

Se um parâmetro pode vir de Path ou Query, Path garante que ele seja **sempre da URL**.

PYTHON

```
@app.get("/pedidos/{pedido_id}")
async def get_pedido(pedido_id: int = Path(), status: str =
"pendente"):
    return {"pedido_id": pedido_id, "status": status}
```

- pedido_id **deve** estar na URL (/pedidos/10)
- status pode ser um **query parameter** (/pedidos/10?status=finalizado)